Clinic Booking API PRD

Oct 22, 2026 · @Ed

Overview

Ship a clinic booking API plus a small web page at a live https URL that passes all five levels of the Koya spec (OpenAPI 3.0.3, v1.0). The spec is the contract: exact paths, status codes and `{"error": "<message>"}` bodies. Anything the spec leaves open is decided in this doc.

In scope

- 6 operations: `POST /reset`, `GET /slots`, `POST /bookings`, `GET /bookings`, `GET /bookings/{id}`, `DELETE /bookings/{id}`
- Web page: browse slots, book one, see the confirmation
- Postgres persistence, so bookings survive between requests and deploys

Out of scope: patient accounts, provider admin UI, emails or notifications, payments.

Done means: every level's checks pass against the live URL, and the deploy stays untouched after submission.

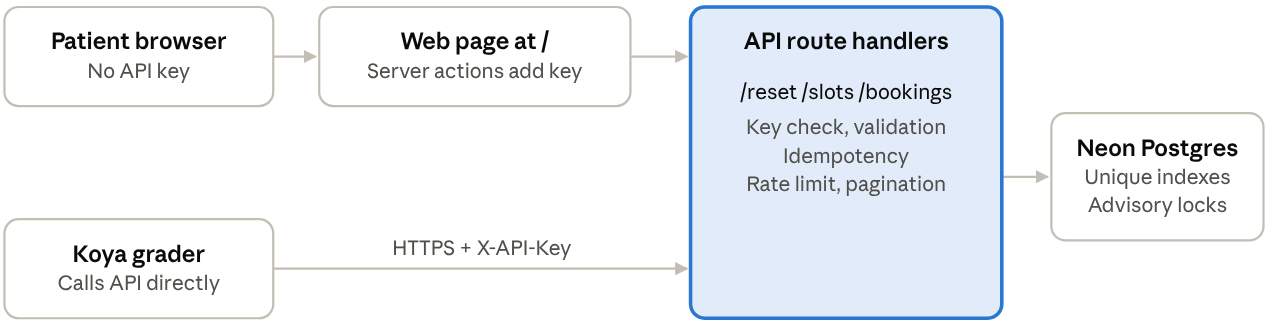
Stack and architecture

One Next.js app on Vercel serves both the API and the page; Neon Postgres stores everything.

Layer	Choice	Why
Framework	TypeScript, Next.js App Router route handlers	API and page in one repo and one deploy
Hosting	Vercel free plan	https out of the box, deploys from Git
Database	Neon Postgres, pooled connection string	Constraints and transactions carry L3 to L5; pooling survives bursts of parallel requests

Layer	Choice	Why
DB driver	`@neondatabase/serverless` Pool	Transactions from serverless functions
Validation	zod	One schema shared by the API and the booking form

The page and the grader hit one API; Postgres enforces the rules



architecture · page, grader, API, database

The grader talks to the API directly with the key; the browser only talks to the page, which adds the key server-side.

Data model

Four Postgres tables. The database enforces the hard rules (one confirmed booking per slot, one row per idempotency key), so correctness never depends on app-level timing.

Table	Columns	Constraints and indexes
`slots`	`id text`, `provider text`, `start_at timestamptz`, `end_at timestamptz`, `start_raw text`, `end_raw text`	PK `id`; index on `(start_at, id)`

Table	Columns	Constraints and indexes
`bookings`	`id text` (UUID), `seq bigserial`, `slot_id text`, `name text`, `email text` (lowercased), `status text`, `created_at timestamptz`	PK `id`; FK `slot_id`; CHECK status in (confirmed, cancelled); **partial unique index on `slot_id` WHERE status = 'confirmed'**; index on `(email, seq)`
`idempotency_keys`	`key text`, `request_hash text`, `status_code int` (null while pending), `response_body jsonb`, `created_at timestamptz`	PK `key`
`rate_limit_attempts`	`email text`, `attempted_at timestamptz`	index on `(email, attempted_at)`

Why `start_raw` and `end_raw`: Postgres plus JS `toISOString()` returns `09:00:00.000Z`, while the spec examples use `09:00:00Z`. Slot times are echoed back exactly as `/reset` received them, so string comparisons in the grader can't fail.

Why `seq`: a strictly increasing number gives a stable "oldest first" order and a clean cursor, with no ties on identical `created_at` values.

API contract

All routes are served at the root (`/slots`, not `/api/slots`), take and return JSON, and require `X-API-Key`. Every non-2xx body is `{"error": "<message>"}`.

Method	Path	Level	Success	Errors
POST	`/reset`	1	200 `{"ok": true, "slots": n}`	400 bad payload, 401
GET	`/slots?provider=`	1	200 `{"slots": [Slot + available]}`	401
POST	`/bookings`	1 to 5	201 Booking (`confirmed`)	400, 401, 404, 409, 422, 429
GET	`/bookings/{id}`	1	200 Booking	401, 404

Method	Path	Level	Success	Errors
DELETE	`/bookings/{id}`	3	200 Booking (`cancelled`)	401, 404, 409 already cancelled
GET	`/bookings?email=&limit=&cursor=`	4	200 `{"bookings": [...], "next_cursor": string or null}`	400 bad limit, missing email, bad cursor; 401

Shapes

- Slot: `id`, `provider`, `start`, `end` (times echoed as received)
- Booking: `id`, `slot_id`, `name`, `email`, `status`, `created_at` (ISO 8601 UTC)

Per-endpoint rules

- `/reset`: one transaction deletes all bookings, slots, idempotency keys and rate limit attempts, then inserts the new slots, past ones included. 400 if `slots` isn't an array, a slot misses a field, a time won't parse, or two slots share an id.
- `/slots`: `provider` is an exact, case-sensitive match. Sorted by `start`, then `id`. `available` is false only while a confirmed booking exists.
- `POST /bookings`: trim `name` and `email`, lowercase `email`. Non-string fields count as invalid (400). Full check order is in the next section.
- `DELETE /bookings/{id}`: sets `cancelled` and frees the slot for rebooking.
- `GET /bookings`: case-insensitive email match, oldest first, includes cancelled bookings. `limit` must be a whole number from 1 to 50 (default 10); `0`, `51`, `abc`, `2.5` all return 400.

Global behaviour

- Missing or wrong key returns 401 on every route, `/reset` included. `OPTIONS` preflight is exempt.
- Unknown route: 404. Wrong method: 405. Malformed JSON: 400. Unhandled exception: 500 `{"error": "internal error"}`. All JSON.
- API key compared in constant time.

POST /bookings: check order

The order decides which status code a grader sees when a request breaks several rules at once, so it is fixed here. The first failing step returns.

1. **API key.** Missing or wrong: 401.
2. **Parse JSON.** Malformed or not an object: 400.
3. **Normalize.** Trim `name` and `email` if they are strings; lowercase `email`.
4. **Idempotency** (only when `Idempotency-Key` is sent).
   - Hash the normalized body.
   - Key exists, same hash, finished: replay the stored status and body. Stop.
   - Key exists, different hash: 422.
   - Key exists, still pending: poll up to about 3s for it to finish, then replay.
   - Key is new: insert a `pending` row and continue.
5. **Rate limit** on the normalized email (skipped when email is missing or empty). Five attempts already in the last 60s: 429 with `Retry-After`.
6. **Validate.** `slot_id`, `name`, `email` present, non-empty strings; email matches a pragmatic pattern (`something@domain.tld`, no spaces): else 400.
7. **Slot exists.** Else 404.
8. **Slot not started.** `start <= now`: 400.
9. **Insert** the booking. Unique index violation: 409. Success: 201.
10. **Store the response** on the idempotency row (any final status except 429), then return it.

Decisions baked into this order

- Replays come before the rate limiter, so retrying a successful request never turns into a 429.
- A 429 is not stored against the idempotency key; the pending row is deleted instead, so the client can retry the same key once the window clears.
- 400, 404 and 409 responses are stored, because the spec says a replay returns the same status code.

Design for levels 3 to 5

Double booking (L3)

The partial unique index on `bookings(slot_id) WHERE status = 'confirmed'` is the whole solution. Twenty parallel inserts for one slot produce exactly one row; the other nineteen hit error code `23505` and return 409. No locks or read-then-write checks in app code.

Cancellation (L3)

One atomic statement: `UPDATE bookings SET status = 'cancelled' WHERE id = $1 AND status = 'confirmed' RETURNING *`. No row back: look the id up. Exists means 409 already cancelled, missing means 404. Once cancelled, the partial index no longer covers the row, so the slot is bookable again.

Idempotency (L4)

- `INSERT ... ON CONFLICT (key) DO NOTHING` claims the key. Whoever inserts the row does the work; everyone else reads it.
- Body hash is SHA-256 of canonical JSON (sorted keys, normalized values), so whitespace or key order differences still count as the same body.
- One API key exists, so keys are global rather than scoped per client.
- Rows are kept until the next `/reset`. No TTL needed at assessment scale.

Rate limiting (L5)

- Sliding 60s window per normalized email, stored in Postgres because serverless instances share no memory.
- In one transaction: take `pg_advisory_xact_lock(hashtext(email))`, count attempts newer than 60s, and either insert a new attempt or return 429. The lock stops two simultaneous requests from both reading a count of 4.
- Every attempt counts (201, 400, 404, 409), except requests rejected with 429 and idempotent replays.
- `Retry-After` = seconds until the oldest attempt in the window turns 60s old, rounded up, minimum 1.
- Other emails are unaffected because the key is the email.

Pagination (L4)

- Order by `seq` ascending. Fetch `limit + 1` rows; the extra row tells us whether another page exists.
- `next_cursor` is the last returned row's `seq`, base64url encoded. Last page returns `next_cursor: null`.
- Cursor that doesn't decode to a positive integer: 400. Unknown email: 200 with `bookings: []` and `next_cursor: null`.

Web page

One page at `/` that patients use to find a slot, book it and see the confirmation. It never holds the API key: it calls server actions (or `/app-api/*` handlers) that add `X-API-Key` server-side and forward to the real API.

Views

1. **Slots:** grouped by day, shown in the visitor's local time, with a provider filter. Booked and already-started slots are visible but disabled.
2. **Book:** name and email form with the same validation as the server. Each form submission generates one `Idempotency-Key` (`crypto.randomUUID()`), so a double click or network retry can't book twice.
3. **Confirmation:** booking id, provider, date and time, status, and a link to look the booking up again.

Error handling

API response	What the patient sees
400	Inline field error with the server's message
404	"That slot no longer exists" and the list refreshes
409	"Someone just booked this slot, pick another" and the list refreshes
429	"Too many attempts, try again in N seconds" using `Retry-After`
5xx or network	Retry button that reuses the same idempotency key

Nice to have (after L5 is solid): a "My bookings" view by email with pagination and a cancel button. It exposes bookings to anyone who knows an email, which is acceptable for the assessment and should be noted in the README.

Testing and acceptance

Every box below is checked against the **deployed** URL, not localhost, with a script (`scripts/smoke.ts`) that calls `/reset` first so each run starts clean.

Level 1

- [ ] `/reset` with past and future slots returns 200; `/slots` returns them sorted by `start`, times byte-identical to what was sent
- [ ] `?provider=Dr Ade` returns only exact matches
- [ ] Booking a future slot returns 201, `status: confirmed`; `GET /bookings/{id}` returns the same booking
- [ ] Unknown booking id returns 404 with `{"error"}`

Level 2

- [ ] No key and wrong key both return 401 on all six operations
- [ ] Missing, empty or whitespace-only `slot_id`, `name`, `email` each return 400; `"ada@"` returns 400
- [ ] `" ADA@Example.com "` is stored as `ada@example.com`; name is trimmed
- [ ] Unknown slot returns 404; a past slot returns 400

Level 3

- [ ] 20 parallel bookings for one slot: exactly one 201, nineteen 409
- [ ] Cancel returns 200 `cancelled`; second cancel returns 409; slot shows `available: true` and can be rebooked

Level 4

- [ ] Same key and body sent twice: same status and same booking id, one row in the DB
- [ ] Same key sent 10 times in parallel: one booking
- [ ] Same key with a different body returns 422
- [ ] 23 bookings for one email at `limit=10`: pages of 10, 10, 3, last `next_cursor` null, no gaps or repeats

- [ ] Uppercase email query finds lowercase bookings; `limit` of 0, 51, `abc`, `2.5` returns 400

Level 5

- [ ] 5 attempts for one email (mix of good and bad) succeed or fail normally; the 6th returns 429 with integer `Retry-After`
- [ ] A different email books fine during that window
- [ ] After waiting `Retry-After` seconds the email can book again
- [ ] `/reset` clears the limit

Web page

- [ ] API key absent from the browser bundle and network tab
- [ ] Double-clicking Book creates one booking
- [ ] Works at phone width

Deploy, submit, freeze

Deploy

- Env vars: `DATABASE_URL` (Neon pooled string), `API_KEY` (32+ random chars).
- Run migrations against Neon before the first deploy; never auto-migrate on request.
- Put the Vercel functions and the Neon database in the same region to keep latency low.
- Neon's free compute pauses when idle, so the first request after a quiet spell is slower. Hit the API once right before submitting.

Submit

- Base URL, API key, repo link.
- README: live URLs, how to run locally, one line per level on how it was met, and the decisions from the check-order section.

Freeze

- Tag the submitted commit `v1.0-submission`.
- Turn off Vercel's Git auto-deploy so a stray push can't change the live build.
- Don't run `/reset` or touch the database after submitting.

Open questions

- Does the grader accept a base URL with a path prefix? Serving at the root avoids the question.
- Should 429 responses count toward the 5-per-minute window? This PRD says no.
- How strict should email validation be? This PRD uses a pragmatic pattern, not full RFC 5322.